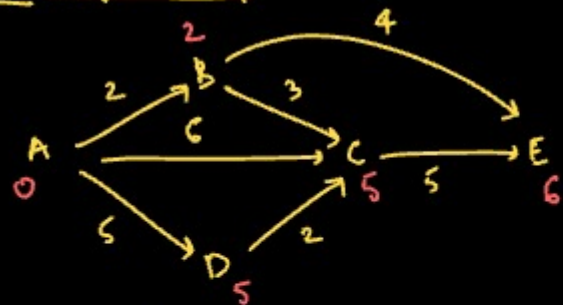


Dynamic Programming



$A \xrightarrow{5} D \xrightarrow{2} C$ length of path = 7

DAG

Directed Acyclic Graph

$f(\text{node}) \rightarrow$ defines the shortest path length from A to that node

$$f(E) = \min \begin{cases} f(B) + 4 \\ f(C) + 5 \end{cases} = \min\{6, 10\} = 6$$

$$f(v) = \min \{ f(u) + c(u,v) \text{ for } u \rightarrow v \}$$

$$f(B) = \min \{ f(A) + 2 \} = 2 \quad f(s) = 0$$

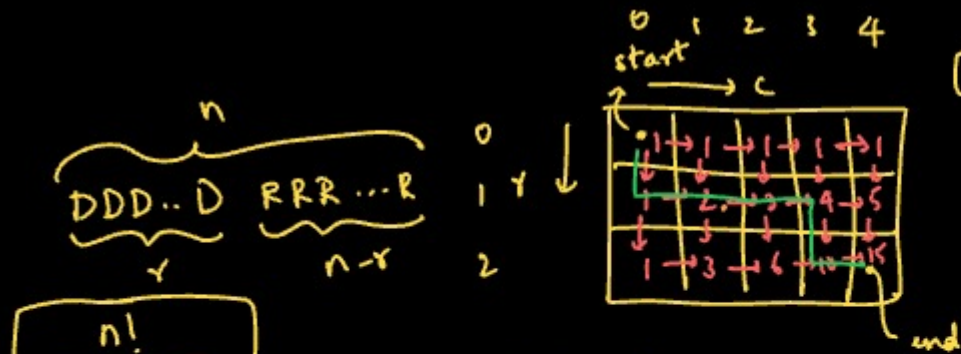
$$f(A) = 0$$

$$f(C) = \min \begin{cases} f(B) + 3 = 5 \\ f(A) + 6 = 6 \\ f(D) + 2 = 7 \end{cases} = 5$$

$$f(D) = \min \{ f(A) + 5 \} = 5$$

• Shortest paths $\rightarrow$ optimal substructure



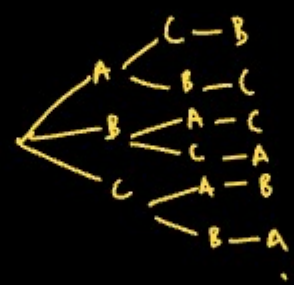


$(1, 2, 3, 4, 5)$ $(ABC) = 3!$
 $3 \times 2 \times 1$

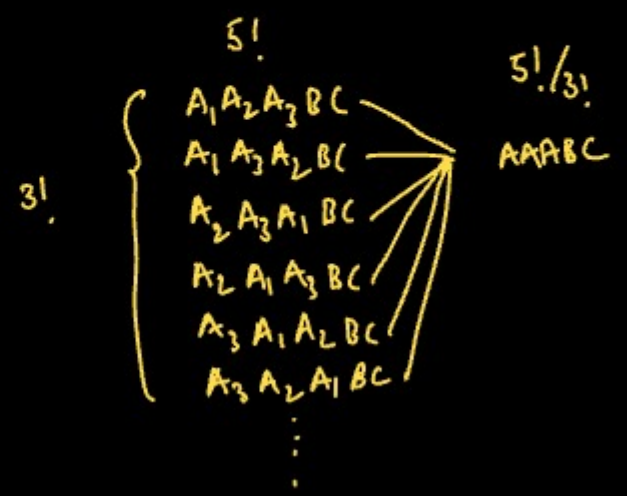
$$\frac{n!}{r!(n-r)!}$$

2 Ds & 4 Rs

$DRRRDR = 6C2$
 $\downarrow$
 $\frac{6!}{2!4!} = {}^6C_2 = \frac{n!}{r!(n-r)!}$



n char string $n!$ permutation
 $AAABBC$ $\frac{6!}{3!2!}$



Steps

1. $f(r, c) := \# \text{unique ways to reach } (r, c) \text{ from the start}$

#parameters = 2

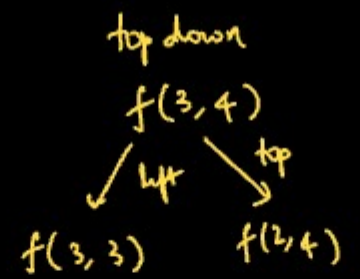
$\Rightarrow$ DP table is 2D

$dp[r][c] := \# \text{unique ways to reach } (r, c) \text{ from } (0, 0)$

2. Base cases: $dp[0][0] = 1$, $dp[r][0] = 1$, $dp[0][c] = 1$

Recurrence relation: $dp[r][c] = \underbrace{dp[r][c-1]}_{\text{left}} + \underbrace{dp[r-1][c]}_{\text{top}}$
 $(r > 0, c > 0)$

3. Order of filling: fill row by row from top to bottom
the table: filling each row from left to right



m rows & n cols

4. Final answer: $dp[m-1][n-1]$

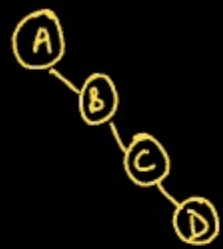
5. R.T.

#subproblems: mn

Work per subproblem: $\Theta(1)$

post processing: $\Theta(1)$

RT = $\Theta(mn)$



$\approx 2^n$ nodes

DP steps

1. Defining a subproblem
2. Base case & Recurrence relation
3. Order of filling the table
4. How do we compute the final solution from the table.
5. Running time:

$$\# \text{subproblems} = f(n)$$

$$\# \text{time per subproblem} = g(n)$$

$$\text{post processing work} = h(n)$$

$$RT = f(n) \cdot g(n) + h(n)$$

Longest Increasing Subsequence

$A = [2, -1, 3, 4, 4, 0, 6]$
 $n = \# \text{ elements}$
length

ABCDE

BCD $\rightarrow$ subseq ✓
subsequence ✓
ACD $\rightarrow$ subseq X
subsequence ✓

1. Subproblem:

$dp[i] := \text{length of the longest subsequence}$
starting at index 'i'

2. Base case & Recurrence relation

$$dp[n-1] = 1$$

$$dp[i] = 1 + \max \{ dp[j] \mid i < j \leq n-1 \text{ and } A[i] < A[j] \}$$

3. Ordering of filling the table

from right to left ($dp[n-1]$ to $dp[0]$)

4. Final answer:

$$\max \{ dp[i] : 0 \leq i < n \}$$

5. #subproblems: } $\Theta(n^2)$
work per subproblem: }
post processing: $\Theta(n)$

Total time: $\Theta(n^2)$

$$\begin{array}{lcl} n-1 & : & 1 \\ n-2 & : & 1 \\ n-3 & : & 2 \\ & \vdots & \\ 1 & : & n-1 \\ 0 & : & n \\ & \approx & \frac{n(n+1)}{2} \end{array}$$

1. Subproblem

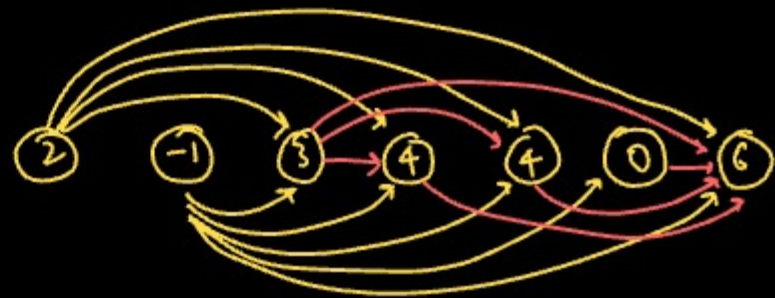
$dp[i]$: Length of LIS
ending at i

2. Base case & recurrence relation

$$dp[0] = 1$$

$$dp[i] = 1 + \max \{ dp[j] : 0 \leq j < i \text{ and } A[j] < A[i] \}$$

3. Fill from left to right



LIS reduces to longest path in a
DAG problem

→ [0, 1, 2, 3, ..., 9]

↪ [i for i in range(10)]

↪ [i² for i in range(1, 11)]

↪ [1, 4, 9, ..., 100]

↪ [i x (i+1) for i in range(1, 7, 2)]

↪ [2, 12, 30]

→ [[0, 0, 0], [1, 1, 1]]

num_rows = 2

num_cols = 3

[[i for j in range(nc)] for i in range(num_rows)]

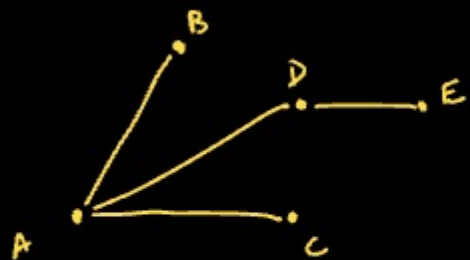
{ 0: "0", 1: "1", 2: "2", ..., 9: "9" }

{ i: str(i) for i in range(10) }

	0	1	2	3	4	5	6	7
A	2	-1	3	4	4	0	16	4
dp	1	1	2	3	3	2	4	3
prev	-	-	0	2	2	1	3	2

2 3 4 16

Weighted Independent Set



Independent set:

Given an undirected graph $G(V, E)$,
an independent set is a subset of V such that
there are no edges among the vertices in the subset.

Independent sets: $\{B, D, C\}, \{\}, \{A\}, \{B\}, \{C\}, \{D\}, \{E\}$
 $\{B, D\}, \{B, C\}, \{B, E\}, \{C, D\}, \{C, E\}$
 $\{B, C, E\}, \{A, E\}$

• # independent sets is exponential in terms of ' n ' = #vertices

• Max weighted Independent Set

Given: an undirected $G(V, E)$

& each vertex has a weight

Output: Find the independent set with
max weight.

Eg: $A \underset{1}{\circ} \overset{B}{\text{---}} \underset{4}{\circ} \overset{C}{\text{---}} \underset{5}{\circ} \overset{D}{\text{---}} \underset{4}{\circ}$

Path graph:

n vertices

& $n-1$ edges



• Given a solution S ,

case 1: x_{n-1} is not part of $S \Rightarrow W[n-2]$

case 2: x_{n-1} is part of $S \Rightarrow w_{n-1} + W[n-3]$

due to optimal substructure

$W[i] \rightarrow$ Max weight IS using $\{0, \dots, i\}$
(MWIS)

1. Subproblem

$dp[i] :=$ weight of MWIS
using vertices $\{x_0, x_1, \dots, x_i\}$

2. Base case

$$dp[0] = w_0$$

$$dp[1] = \max\{w_0, w_1\}$$

Recurrence relation:

$$dp[i] = \max\{dp[i-1], w_i + dp[i-2]\}$$

3. Order of filling:

from left to right

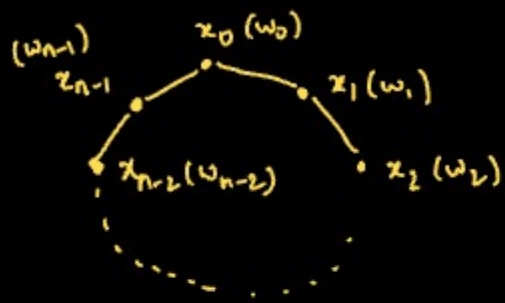
4. Final answer: $dp[n-1]$

5. #subproblems: n

work per subproblem: $\Theta(1)$

postprocessing: $\Theta(1)$

$\rightarrow$ RT: $\Theta(n)$, space:



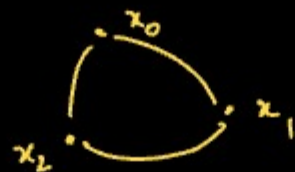
Cycle graph

$dp[i] = \max \text{ vertices using } \{x_0, x_1, \dots, x_i\}$
in the original graph

Given an optimal solution S :

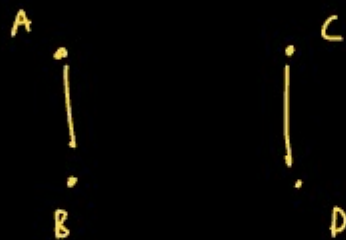
Case 1: doesn't contain $x_{n-1} \rightarrow$ MWIS in path graph
 $x_0 - x_1 - \dots - x_{n-2}$

Case 2: does contain $x_{n-1} \rightarrow$ MWIS in path graph
 $x_1 - x_2 - x_3 - \dots - x_{n-4} - x_{n-3}$
 $\uparrow$
 w_{n-1}

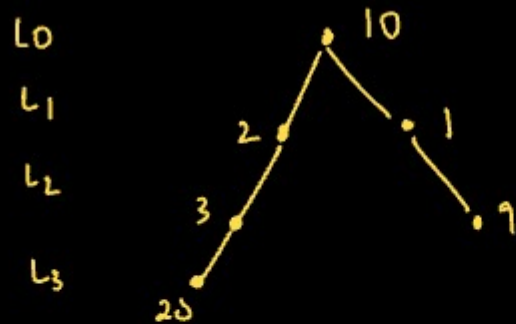


$HRI(\{x_0, x_1\})$

$HRI(\{x_2\})$



MWIS
on trees



optimal
Given an solution S ,

Case 1: doesn't contain root $\Rightarrow S = \text{MWIS}(\text{root.left}) + \text{MWIS}(\text{root.right})$

Case 2: does contain root $\rightarrow S = \text{sum} \{ \text{MWIS}(\text{root.grandchild}) \} + \text{root.val}$

wt of
 $\text{dp-i}(\text{root}) := \text{MWIS of subtree including its root}$
 $\text{dp-ni}(\text{root}) := \text{MWIS of subtree excluding its root}$

$$\text{dp-i}(\text{root}) = \text{dp-ni}(\text{root.left}) + \text{dp-ni}(\text{root.right}) + \text{root.val}$$

$$\text{dp-ni}(\text{root}) = \max \{ \text{dp-i}(\text{root.left}), \text{dp-ni}(\text{root.left}) \} + \max \{ \text{dp-i}(\text{root.right}), \text{dp-ni}(\text{root.right}) \}$$

$$\text{Final answer} = \max \{ \text{dp-i}(\text{original-root}), \text{dp-ni}(\text{original-root}) \}$$

$$\text{dp-i}(\text{empty tree}) = 0$$

$$\text{dp-ni}(\text{empty tree}) = 0$$

R.T:

#subproblems: $2n$

work per subproblem: $\Theta(1) \Rightarrow \text{RT: } \Theta(n)$

post processing: $\Theta(1)$

[2, 3, 1, 4, 9, 6, 2, 8]

Invariants

1. $dp[i]$: The smallest # that ends a increasing subsequence of length " $i+1$ " among the numbers seen so far.

2. $len(dp)$: length of the LIS among the numbers seen so far

[2, 3, 1, 4, 9, 6, 2, 8]

2	3	<u>4</u>	9	8
<u>1</u>	<u>2</u>		<u>6</u>	

[2, 4, 4, 6, 6, 6, 8, 8, 10]

1. index of largest # $\leq$ key

index of smallest # $\geq$ key

lower-bound

upper-bound ...

- Record potential solutions along the way & keep improving the solutions until we run out of them (we'll make sure to never eliminate better solutions)

Candidate Variable
Pattern

solution = 6

key = 8

Min coins:

$[1, 2, 5, 10] \rightarrow$ denominations

$$24 \xrightarrow{2 \times 10} 4 \xrightarrow{2 \times 2} 0 \quad 4 \text{ coins}$$

$[1, 8, 10]$

$$24 \xrightarrow{2 \times 10} 4 \xrightarrow{4 \times 1} 0 \quad 6 \text{ coins}$$

$S: c_1, c_2, c_3, \dots, c_k \quad \text{min coins} = k$

$$\begin{array}{c} \downarrow \\ c_k = 10 \quad \underbrace{\text{min-coins}(n-10) + 1}_{k-1} \end{array}$$

$dp[i]$: min coins required to form 'i' using given denominations

Target: n

denominations: $d_1, d_2, \dots, d_m$

Base case: $dp[0] = 0$

Recurrence relation: $dp[i] = 1 + \min \{ dp[i - d_j] \mid \forall \text{ denomination } d_j \text{ s.t. } d_j \leq i \}$

Order of filling the table: left to right

Final answer: $dp[n] \rightarrow$ size of dp array is ' $n+1$ '

Running time: #subproblems: $n+1$
work per subproblem: $\Theta(m)$

postprocessing: $\Theta(1)$

$\Rightarrow$ RT: $\Theta(nm)$

Pseudo polynomial

n requires ' p ' bits to represent

input size = p

$$p = \lfloor \lg n \rfloor + 1 \quad n \sim 2^p$$

$$= \Theta(2^p m)$$

$$8 = 1000$$

$$16 = 10000$$

$$15 = 1111$$

$$\lg \equiv \log_2$$

$$\ln \equiv \log_e$$

Longest Common Subsequence between 2 strings

AABCADE
AFAAEDC

A B C D F
 A E C G D

↓

A B - C - D F
 A - E C G D -

=

Last column: F or -
 ↓
 LCS (ABCD, AECGD) = 3
 ↓
 LCS (ABCDF, AECG) = 2

2D array

$r0$ $r1$ $r2$ $r3$

.	.	.	.
.	.	.	.
.	.	.	.
.	.	.	.

$A[m][n]$

↓

↓

↓

↓

$dp(i)$: MWIS using $x_0, x_1, \dots, x_i$

Order: left to right

$dp(i)$: MWIS using $x_i, x_{i+1}, \dots, x_{n-1}$

Order: right to left

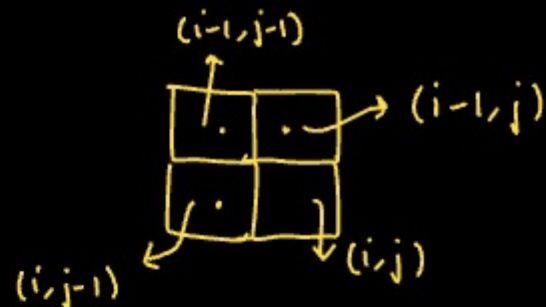
Problem: Find length of the
 LCS between s_1 & s_2
 ↓
 m chars
 ↓
 n chars

1. $dp[i][j]$: Length of LCS
 between $s_1[:i] \rightarrow$ first i chars of s_1
 & $s_2[:j] \rightarrow$ first j chars of s_2

#rows = $m+1$ (i goes from 0 to m)
 #cols = $n+1$
 $(m+1) \times (n+1)$

2. Base case: $dp[i][0] = 0$
 $dp[0][j] = 0$

Recurrence: $dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{if } s_1[i-1] == s_2[j-1] \\ \max\{dp[i-1][j], dp[i][j-1]\} & \text{otherwise} \end{cases}$



3. Order of filling:

row by row filling each row from left to right

4. Final answer: $dp[m][n]$

5. RT: #subproblems: $(m+1)(n+1)$

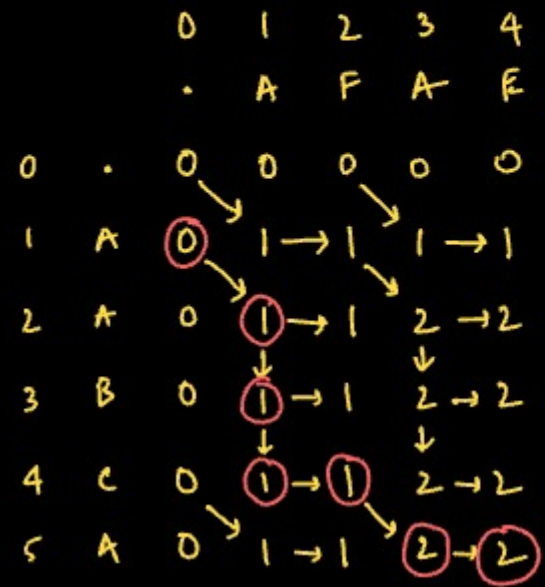
work per subproblem: $\Theta(1)$

postprocessing: $\Theta(1)$

RT: $\Theta(mn)$

$S_1: AABCA \Rightarrow m=5$

$S_2: AFAE \Rightarrow n=4$



AA